```python
#!/usr/bin/env python3
"""
malper-analyse — Vulnerability report analyser & attack map generator
Uses OpenRouter API to reanalyse scanner output and produce clean Obsidian markdo
"""

import argparse
import json
import os
import sys
import textwrap
from datetime import datetime
from pathlib import Path

# ──────────────────────────────────────────────────────────────
# Colour helpers (no deps)
# ──────────────────────────────────────────────────────────────
R  = "\033[1;31m"
G  = "\033[1;32m"
Y  = "\033[1;33m"
C  = "\033[1;36m"
B  = "\033[1;34m"
M  = "\033[1;35m"
W  = "\033[0m"
DIM = "\033[2m"

BANNER = f"""{C}
  ████████╗ ██████╗  ██████╗ ██╗      ██████╗  ██████╗ ██╗  ██╗
  ╚══██╔══╝██╔═══██╗██╔═══██╗██║     ██╔════╝ ██╔═══██╗╚██╗██╔╝
     ██║   ██║   ██║██║   ██║██║     ██║  ███╗██║   ██║ ╚███╔╝
     ██║   ██║   ██║██║   ██║██║     ██║   ██║██║   ██║ ██╔██╗
     ██║   ╚██████╔╝╚██████╔╝███████╗╚██████╔╝╚██████╔╝██╔╝ ██╗
     ╚═╝    ╚═════╝  ╚═════╝ ╚══════╝ ╚═════╝  ╚═════╝ ╚═╝  ╚═╝{W}
{DIM}  malper-analyse  ·  pentest report validator & obsidian attack mapper{W}
"""

DEFAULT_MODEL = "openrouter/auto"
OPENROUTER_URL = "https://openrouter.ai/api/v1/chat/completions"
ENV_KEY_NAME   = "OPENROUTER_API_KEY"
ENV_MODEL_NAME = "MALPER_MODEL"
ENV_FILE       = os.path.expanduser("~/.bashrc")   # also writes to .profile


# ──────────────────────────────────────────────────────────────
```

```
# API key management
# ──────────────────────────────────────────────────────────────────
def get_api_key() -> str:
    key = os.environ.get(ENV_KEY_NAME)
    if key:
        return key

    print(f"\n{Y}[!]{W} No {C}{ENV_KEY_NAME}{W} found in environment.\n")
    print(f"    Get one free at {B}https://openrouter.ai/keys{W}\n")
    key = input(f"    {G}Paste your OpenRouter API key:{W} ").strip()

    if not key:
        print(f"{R}[✗]{W} No key supplied. Exiting.")
        sys.exit(1)

    _persist_env(ENV_KEY_NAME, key)
    os.environ[ENV_KEY_NAME] = key
    return key


def _persist_env(name: str, value: str):
    """Append export line to ~/.bashrc and ~/.profile so it survives reboots."""
    export_line = f'\nexport {name}="{value}"\n'
    for rc in [os.path.expanduser("~/.bashrc"), os.path.expanduser("~/.profile")]:
        try:
            text = Path(rc).read_text() if Path(rc).exists() else ""
            if name in text:
                # Replace existing value
                lines = text.splitlines(keepends=True)
                new_lines = []
                for line in lines:
                    if line.startswith(f"export {name}="):
                        new_lines.append(f'export {name}="{value}"\n')
                    else:
                        new_lines.append(line)
                Path(rc).write_text("".join(new_lines))
            else:
                with open(rc, "a") as f:
                    f.write(export_line)
            print(f"{G}[✓]{W} Saved {name} to {rc}")
        except Exception as e:
            print(f"{Y}[!]{W} Could not write to {rc}: {e}")


# ──────────────────────────────────────────────────────────────────
# File loading
# ──────────────────────────────────────────────────────────────────
```

```
def load_report(path: Path) -> str:
    suffix = path.suffix.lower()
    if suffix not in {".json", ".md"}:
        print(f"{R}[✗]{W} Unsupported file type '{suffix}'. Only .json and .md ar
        sys.exit(1)
    if not path.exists():
        print(f"{R}[✗]{W} File not found: {path}")
        sys.exit(1)

    raw = path.read_text(encoding="utf-8", errors="replace")
    if suffix == ".json":
        try:
            parsed = json.loads(raw)
            return json.dumps(parsed, indent=2)
        except json.JSONDecodeError as e:
            print(f"{Y}[!]{W} JSON parse warning: {e}  — sending raw text anyway.
    return raw


# ─────────────────────────────────────────────────────────────
# OpenRouter call
# ─────────────────────────────────────────────────────────────
SYSTEM_PROMPT = textwrap.dedent("""
You are an elite penetration tester and red team operator reviewing an automated
Automated scanners frequently over-report severity — they cannot understand actua
or whether a "critical" finding is behind layers of authentication or network con

Your job:
1. **Critically reanalyse** every finding. Downgrade severity where the scanner i
2. **Remove false positives** — if a finding has no realistic exploit path, flag
3. **Validate CVEs / CWEs** — only include them if they are real and match the de
4. **Produce a clean, obsidian-compatible markdown report** with the structure de
5. Always think like an attacker. For each real vuln ask: "Can I actually exploit

---

OUTPUT FORMAT (strictly follow this):

# 🛡️ Vulnerability Analysis Report
> Auto-generated by malper-analyse | {timestamp}

## Executive Summary
One paragraph plain-English summary for a non-technical audience. Include overall

## Risk Score
| Metric | Value |
|---|---|
```

```
| Critical | N |
| High | N |
| Medium | N |
| Low | N |
| False Positives Removed | N |
| Overall Risk | Critical / High / Medium / Low |

## Validated Vulnerabilities

For EACH real vulnerability use this card format:

### [SEVERITY EMOJI] CVE-XXXX-XXXX — Short Title
*(use 🔴 Critical, 🟠 High, 🟡 Medium, 🔵 Low, ⚪ Info)*

| Field | Detail |
|---|---|
| **Severity** | Critical / High / Medium / Low |
| **CVSS** | X.X |
| **CVE** | CVE-XXXX-XXXX or N/A |
| **CWE** | CWE-XXX |
| **Affected Component** | |
| **Exploitability** | Easy / Moderate / Hard |
| **Auth Required** | Yes / No |

**Description**
Two to three sentences explaining what the vuln actually is.

**Real-World Impact**
What an attacker can actually achieve if exploited.

**Proof of Concept Sketch**
High-level steps or payload skeleton. DO NOT write full working exploits.

**Remediation**
Concrete fix. Version to upgrade to, config to change, code pattern to fix.

**Tags**
`#vuln/[category]` `[[Component Name]]` `[[CVE-XXXX-XXXX]]`

---

## Removed / Downgraded Findings
List findings the scanner flagged that were removed or downgraded, with a one-lin

| Original Finding | Original Severity | Action | Reason |
|---|---|---|---|
```

```
---

## 🗺️ Attack Map

> Obsidian graph-compatible. Each node is a `[[wikilink]]`. Copy this section int

```mermaid
graph TD
    A[🌐 External Attacker] --> B{Initial Access}
    B --> |Exploit [[CVE-XXXX-XXXX]]| C[Foothold]
    C --> D{Lateral Movement}
    D --> E[Privilege Escalation]
    E --> F[💥 Impact: Data Exfil / RCE / Persistence]
```

Obsidian Node Links
List every exploitable component as a wikilink with a one-line annotation:
- `[[Component]]` — reason it's a node in the attack path

Analyst Notes
Any additional observations, things to manually verify, or scanner quirks noticed

---

*Report generated by [malper-analyse](https://github.com/MKMithun2806) · Model: {
""")

def call_openrouter(api_key: str, model: str, report_content: str, filename: str)
    import urllib.request

    ts = datetime.now().strftime("%Y-%m-%d %H:%M")
    system = SYSTEM_PROMPT.replace("{timestamp}", ts).replace("{model}", model)

    user_msg = f"Here is the vulnerability scanner report from file `{filename}`.

    payload = json.dumps({
        "model": model,
        "messages": [
            {"role": "system", "content": system},
            {"role": "user",   "content": user_msg},
        ],
        "temperature": 0.3,
        "max_tokens": 8192,
    }).encode("utf-8")
```

```
    req = urllib.request.Request(
        OPENROUTER_URL,
        data=payload,
        headers={
            "Authorization": f"Bearer {api_key}",
            "Content-Type":  "application/json",
            "HTTP-Referer":  "https://github.com/MKMithun2806/malper-analyse",
            "X-Title":       "malper-analyse",
        },
        method="POST",
    )

    print(f"{C}[→]{W} Sending to {B}{model}{W} via OpenRouter…")
    try:
        with urllib.request.urlopen(req, timeout=120) as resp:
            body = json.loads(resp.read().decode("utf-8"))
    except urllib.error.HTTPError as e:
        err_body = e.read().decode("utf-8", errors="replace")
        print(f"{R}[✗]{W} HTTP {e.code}: {err_body}")
        sys.exit(1)
    except Exception as e:
        print(f"{R}[✗]{W} Request failed: {e}")
        sys.exit(1)

    try:
        return body["choices"][0]["message"]["content"]
    except (KeyError, IndexError):
        print(f"{R}[✗]{W} Unexpected response schema:\n{json.dumps(body, indent=2
        sys.exit(1)


# ─────────────────────────────────────────────────────────────────
# Output
# ─────────────────────────────────────────────────────────────────
def write_output(input_path: Path, content: str) -> Path:
    ts = datetime.now().strftime("%Y%m%d_%H%M%S")
    stem = input_path.stem
    out_path = input_path.parent / f"{stem}_analysed_{ts}.md"
    out_path.write_text(content, encoding="utf-8")
    return out_path


def print_summary(out_path: Path, content: str):
    lines = content.splitlines()
    crit  = sum(1 for l in lines if "🔴" in l)
    high  = sum(1 for l in lines if "🟠" in l)
```

```
    med   = sum(1 for l in lines if "🟡" in l)
    low   = sum(1 for l in lines if "🔵" in l)
    fps   = 0
    for l in lines:
        if "False Positives Removed" in l:
            parts = l.split("|")
            for p in parts:
                p = p.strip()
                if p.isdigit():
                    fps = int(p)
                    break

    print(f"\n{G}{'─'*55}{W}")
    print(f"  {G}[✓]{W} Analysis complete!")
    print(f"  {G}[✓]{W} Report saved → {C}{out_path}{W}")
    print(f"\n  {R}●{W} Critical  {crit:>3}    {Y}●{W} Medium  {med:>3}")
    print(f"  {M}●{W} High      {high:>3}    {B}●{W} Low     {low:>3}")
    if fps:
        print(f"  {DIM}✂  {fps} false positive(s) removed by analyst{W}")
    print(f"{G}{'─'*55}{W}\n")


# ──────────────────────────────────────────────────────────────
# Entry point
# ──────────────────────────────────────────────────────────────
def main():
    print(BANNER)

    parser = argparse.ArgumentParser(
        prog="malper-analyse",
        description="Reanalyse a vulnerability scanner report and generate a clea
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=textwrap.dedent(f"""
        examples:
          malper-analyse report.json
          malper-analyse nmap_vulns.md --model anthropic/claude-opus-4-5
          malper-analyse scan.json --output /tmp/

        env vars:
          {ENV_KEY_NAME}    OpenRouter API key (prompted if missing)
          {ENV_MODEL_NAME}  Override default model ({DEFAULT_MODEL})
        """),
    )
    parser.add_argument("report", metavar="FILE", help=".json or .md vulnerabilit
    parser.add_argument("--model", "-m", default=None, help="OpenRouter model str
    parser.add_argument("--output", "-o", default=None, help="Output directory (d
    parser.add_argument("--no-banner", action="store_true", help="Suppress the ba
```

```
    args = parser.parse_args()

    input_path = Path(args.report).expanduser().resolve()

    # Resolve output path
    if args.output:
        out_dir = Path(args.output).expanduser().resolve()
        out_dir.mkdir(parents=True, exist_ok=True)
    else:
        out_dir = input_path.parent

    model = (
        args.model
        or os.environ.get(ENV_MODEL_NAME)
        or DEFAULT_MODEL
    )

    api_key = get_api_key()

    print(f"{G}[✓]{W} Loading {C}{input_path.name}{W}…")
    report_content = load_report(input_path)
    size_kb = len(report_content.encode()) / 1024
    print(f"{G}[✓]{W} Read {size_kb:.1f} KB  ·  model: {B}{model}{W}")

    result = call_openrouter(api_key, model, report_content, input_path.name)

    # Redirect output if custom dir
    final_input_path = out_dir / input_path.name
    out_path = write_output(Path(str(final_input_path)), result)

    print_summary(out_path, result)


if __name__ == "__main__":
    main()
```